# Factor-base designs: L2 rerun under a limit the host honours, L2r (run-20261019T1740Z)

**Kind:** solver-axis stage diagnostic (AGENTS.md §2). Observations only. Nothing here prices a pipeline, reports S or shows a speedup. The protocol is `PREREGISTRATION.md` Amendment 6, registered before any L2r row (commit `b39f8b647`). The analysis is `runs/analysis-amendment6.txt` (native `analyze --amendment6`). Every number below is copied from it, from the row files or from `runs/L2r-A11-m4-t5.dmesg.txt`. Amendment 5′s run is `reports/run-20261019T1633Z`.

## Run

- **Pre-flight** (2026-10-19T17:40:39Z). Memory cgroup `/process_api/01a1552f-…/claude-code-bash`, `memory.limit_in_bytes` 14 343 688 192 B = 14 007 508 kB. `MemTotal` 16 479 424 kB, `nproc` 4. That gives V = min(14 500 000, 14 007 508 − 524 288, 16 479 424 − 524 288) = **13 483 220 kB**. The kernel log had no OOM lines before the first row.
- **Command** (`B=target/release/examples/fb_designs_slope`, `R=research/factor_base_designs_20261015/runs`, detached, one system at a time):
  ```
  $B run --arms A11 --m 4 --t-min 5 --t-max 5 --budget 14400 --grace 600 --vmem-kb 13483220 --out-dir $R --file 'L2r-{arm}-m4-t5'
  $B analyze $R --amendment6 > $R/analysis-amendment6.txt
  ```
- **Binary:** `2cdebec95`. This adds the `L2r` row group and `analyze --amendment6` to L2′s harness (`4aec76a5a`). The run path is unchanged. `analyze --amendment2`, `--amendment4` and `--amendment5` reproduce their committed outputs byte for byte, and `clippy -D warnings` is clean.
- **Host:** "vm", Intel Xeon @ 2.80 GHz, 4 cores, a Claude Code cloud container. This container was booted 17:21:47Z, so it is not the one L2 ran in, although the CPU label, `MemTotal` and cgroup limit are the same. The host was otherwise idle. **Window:** 17:40:40Z–17:45:01Z.
- **Records:** `runs/L2r-A11-m4-t5.{jsonl,log}`, `runs/L2r-A11-m4-t5.dmesg.txt` (pre-flight values and the kernel-log check), `runs/analysis-amendment6.txt`.

## L2r: A11, m = 4, t = 5 (b = 5, |V| = 32, 42 variables, 43 equations)

| system | status | time (s) | peak RSS (kB) | stop | limit |
|---|---|---|---|---|---|
| target 0 | aborted | 130.0 | 13 311 580 | malloc failed under `ulimit -v 13483220` (SIGABRT) | attributable |
| target 1 | aborted | 131.8 | 13 286 088 | malloc failed under `ulimit -v 13483220` (SIGABRT) | attributable |

- **Kernel log:** no OOM lines after the series either. Both limits are therefore the registered one.
- **Identity with L2:** 2 of 2 systems match on cell seed, target seeds, `n_vars`, `n_eqs`, planted x, $x_R$, curve and set.
- **L2 for comparison** (Amendment 5, under `ulimit -v 14500000`): both systems were killed by the memory-cgroup OOM killer at 97.3 s and 108.0 s, with peak RSS 13 974 324 and 13 974 336 kB.
- No `field_ops`, D or `max_cols` exist, because a memory abort leaves no F4 counters. The partial values are wall time and peak RSS.

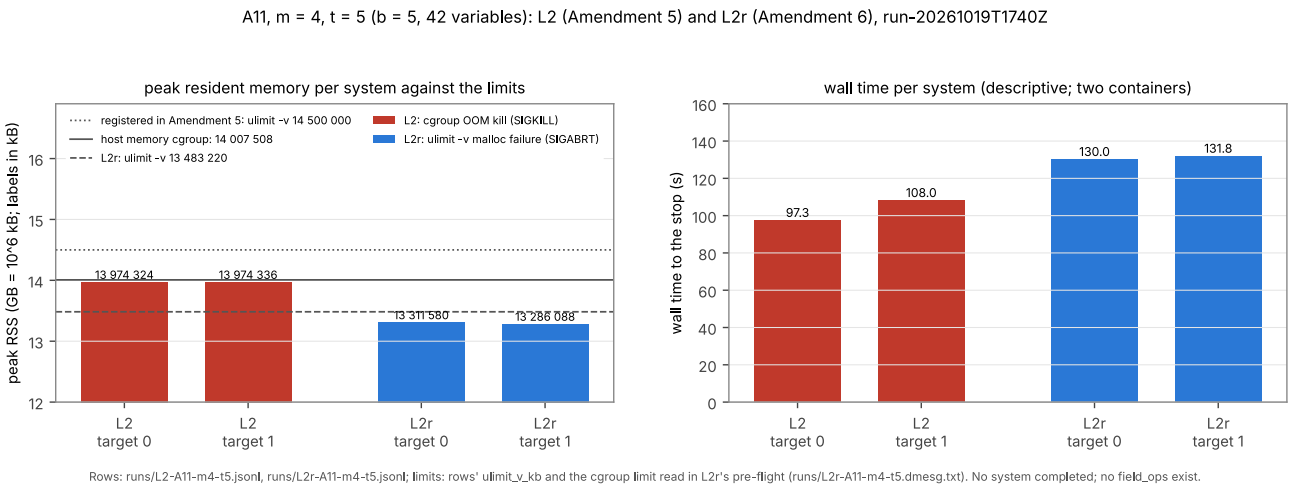



Figure 1: Left: peak RSS of each A11 m = 4, t = 5 system, against L2r's `ulimit -v` (dashed), the host's memory cgroup (solid) and the 14.5 GB that Amendment 5 registered (dotted). Right: wall time to the stop. This is descriptive only: L2 and L2r ran in two different containers. Rows: `runs/L2-A11-m4-t5.jsonl`, `runs/L2r-A11-m4-t5.jsonl`. Limits from the rows and `runs/L2r-A11-m4-t5.dmesg.txt`. Run run-20261019T1740Z, `plot.py`.

## Decision rule (R2 applied to L2r)

**A11 not measurable at m = 4, t = 5 within 13 483 220 kB and 4 h.** Both limits are attributable. A11′s 3-cell m = 4 verdicts stand:

| series | completed cells | κ | σ | verdict |
|---|---|---|---|---|
| A11 m=4 $p_1$ (F4 via C1, C2 t = 4; L2r t = 5 not completed) | 3 (t = 2, 3, 4) | 14.886 | 1.5000 | $\kappa \geq m - 1 = 3$: cost-unfavourable; $\sigma \geq 0.5$: no middle ground |

Amendment 5′s L2 prediction keeps its recorded outcome (**not decided as registered**). Amendment 5′s R1 outcome is unchanged (A0 not measurable at t = 4 within 14.5 GB and 4 h).

## Prediction scorecard (Amendment 6)

| prediction part | observed | outcome |
|---|---|---|
| both systems abort under `ulimit -v V` (malloc failure) | yes, both | as predicted |
| no OOM kill | none in the kernel log | as predicted |
| each before its own L2 kill time (97.3 s, 108.0 s) | 130.0 s and 131.8 s, after | not as predicted |
| **whole prediction** | | **did not hold** |

The prediction used wall time as a stand-in for memory growth. It assumed each L2r system would pass V sooner than its L2 counterpart reached 13.97 GB. Instead, both L2r systems reached a lower peak (13.3 GB) later (130 s, against 97–108 s). The two runs were in different containers. This run did not measure the cause of the difference: CPU share, memory-allocation behaviour under the different limits, or host noise.

## Limits and what is missing

- The rerun gives no cost point. Whether A11 at m = 4, t = 5 completes within 14.5 GB is still unmeasured: this study has no host with a memory cgroup of at least 15 GB.
- Two systems, one host, no isolation receipt. Wall times are descriptive only, and they are not comparable across the two containers.